



Escuela de Ingeniería y Tecnologías

Proyecto de Introducción a la Ingeniería

Avances del Proyecto 3

Profesor: Antonio Martinez Torteya.

Geú Dávila Guerrero 656528
Angel Gabriel Romero Patena 694550
Esaúl Alejandro Villarreal Ábrego 646239

Este trabajo utilizó el apoyo de la inteligencia artificial de Google Gemini, limitado a la explicación de conceptos y problemas encontrados en el proceso de desarrollo (sin haberse especificado el código exacto, solo el concepto o problema general).

“Doy mi palabra que he realizado esta actividad con integridad académica.”

- Esaúl Alejandro Villarreal Ábrego
- Ángel Gabriel Romero Patena
- Geú Dávila Guerrero

Introducción

El objetivo de este trabajo es explorar el uso del módulo Bluetooth HC-05 para permitir comunicaciones entre una computadora y un Arduino. Esto permitiría controlar un rover desde la distancia y obtener datos sin la limitación del cable de conexión entre el Arduino y la computadora. Las capacidades que serán demostradas en esta tarea son:

- Controlar el LED integrado de un Arduino haciendo uso de comandos enviados desde una computadora que no es a la cual el microcontrolador está conectado.

Para comenzar, se explicará brevemente el funcionamiento de Bluetooth, así como la manera en que funciona el módulo HC-05. También se realizará un análisis de las mejoras, dificultades, problemas y soluciones encontrados durante el proceso de desarrollo.

Teoría

Bluetooth

Consiste en una tecnología que permite transmitir información a corta distancia de manera inalámbrica. Permite una comunicación eficiente y segura que no gasta mucha energía, lo cual lo hace ideal para una gran variedad de productos que utilizan baterías. El alcance de comunicación entre dos dispositivos que usan bluetooth suele ser de 10 metros (*¿Qué es Bluetooth®?, s.f.*).

La frecuencia de operación es de 2.4 GHz, con múltiples canales disponibles para que los dispositivos conectados puedan constantemente transferirse de uno a otro para encontrar la menor interferencia y mejor calidad de señal posibles (*¿Cómo funciona la tecnología Bluetooth®, s.f.*).

Módulo HC-05

Este módulo Bluetooth puede ser utilizado para múltiples aplicaciones que requieren comunicación inalámbrica, ya sea en configuración maestra (donde busca conectarse con otros dispositivos esclavos de manera activa) o esclava (donde espera que otros dispositivos maestros se conecten a él). Opera a un voltaje de 3.3V (regulado de un voltaje suministrado de 5V) y tiene un rango de 10 metros. Utiliza el protocolo Bluetooth 2.0 y se puede comunicar con un Arduino utilizando UART (con un rango de 9600-460800 baudios). Así mismo, el módulo opera ya sea en *Command Mode* (donde es configurado usando comandos seriales) o *Data Mode* (donde transmite y recibe datos por medio de Bluetooth) (HC-05 bluetooth module interfacing with arduino, s.f.).

Los 6 pines del módulo son:

1. Enable/Key: Controla el estado de poder del módulo (lo puede apagar electrónicamente).
2. VCC: conectado a 5V.
3. GND: conectado a tierra (0V).
4. TXD: conectado al pin RX (configurado) del Arduino.
5. RXD: conectado al pin TX (configurado) del Arduino.
6. State: permite determinar el estado de conexión del módulo.

(HC-05 bluetooth module interfacing with arduino, s.f.)

Metodología

Electrónica

En la **Figura 1** se muestra la referencia utilizada para crear el circuito de la tarea.

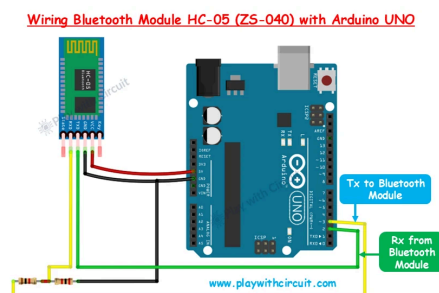


Figura 1: Circuito de referencia utilizado. [Imagen]. s.f. de Play with Circuit (<https://playwithcircuit.com/hc05-bluetooth-module-arduino-tutorial/>)

Al analizar el circuito, se puede identificar que para el pin RXD se utiliza un divisor de voltaje para convertir un voltaje de 5V, suministrado por el pin TX configurado en el Arduino, a 3.3V. Para lograr esto, se utilizan dos resistores con resistencias de 1 kΩ (conecta pin digital a pin RXD) y 2 kΩ (conecta pin RXD a tierra para completar un circuito). Tomando en cuenta que los resistores están conectados en serie y que, en tal caso, el voltaje total equivale a la suma de voltajes en cada componente del sistema, se realiza el siguiente cálculo para confirmar que el pin RXD recibe 3.3V (Series circuits and the application of Ohm's law, s.f.).

$$V = \left(\frac{1000 \Omega}{3000 \Omega} \right) \times 5 V = 1.67 V$$

A través del resistor de 1 kΩ, se ve una caída de 1.67 V en el circuito. De 5 V alimentados al circuito, el pin RXD recibe 3.33 V.

A su vez, no se hace uso de los pines 0 y 1 (RX y TX respectivamente) dado a que interfieren con la comunicación USB del Arduino con la computadora. En cambio, se configuran (ver sección de **Programación**) los pines digitales 10 y 11 para que actúen como

los pines RX y TX, respectivamente, mediante los cuales el módulo HC-05 se comunica con el Arduino (HC-05 bluetooth module interfacing with arduino, s.f.).

Programación

Se utilizaron dos programas: uno para configurar el módulo y otro para permitir la comunicación Bluetooth. El código utilizado para configurar el módulo se muestra en la **Figura 2**.

```
#include <SoftwareSerial.h> // Incluimos la librería SoftwareSerial
SoftwareSerial BT(10,11); // Definimos los pines RX y TX del Arduino conectados al Bluetooth

void setup()
{
  BT.begin(9600); // Inicializamos el puerto serie BT (Para Modo AT 2)
  Serial.begin(9600); // Inicializamos el puerto serie
}

void loop()
{
  if(BT.available()) // Si llega un dato por el puerto BT se envía al monitor serial
  {
    Serial.write(BT.read());
  }

  if(Serial.available()) // Si llega un dato por el monitor serial se envía al puerto BT
  {
    BT.write(Serial.read());
  }
}
```

Figura 2: Código para configurar el módulo. [Captura de pantalla]. s.f. de Naylamp Mechatronics

(https://naylampmechatronics.com/blog/24_configuracion-del-modulo-bluetooth-hc-05-usando-comandos-at.html)

Como se puede observar, se utiliza la librería SoftwareSerial para configurar pines RX y TX para comunicación Bluetooth y para leer y escribir datos al módulo. Con base en este código, es posible usar comandos AT para configurar al HC-05. Cabe destacar que el módulo tiene que estar en el estado AT2, el cual es obtenido al mantener presionado el botón incluido en el HC-05 mientras este es encendido (Configuración del módulo bluetooth HC-05 usando comandos AT, s.f.).

Usando los comandos *AT*, *AT+NAME* y *AT+PSWD* es posible determinar el estado, nombre y contraseña de conexión del módulo respectivamente. Para cambiar el nombre o contraseña se escriben los comandos, seguidos por = *[Nombre/contraseña deseados]*. Una vez cambiada la configuración se debe reescribir el comando (sin el = *[nombre/contraseña]*) para verificar que se hayan registrado correctamente los cambios realizados (Configuración del módulo bluetooth HC-05 usando comandos AT, s.f.).

Una vez que se haya configurado el módulo, se debe programar para poder establecer comunicación bluetooth con otro dispositivo. El programa de referencia utilizado se ve a continuación en la **Figura 3**.

```

/*
Interfacing HC-05 (ZS-040) Bluetooth Module with Arduino UNO using pin 2(Rx) and 3(Tx)
by www.playwithcircuit.com
*/

// Include Software Serial Library this library makes DIO pins as Serial Pins
#include <SoftwareSerial.h>

//Create software serial object to communicate with HC-05
SoftwareSerial BTSerial(2, 3); // Now pin 2 and pin 3 of Arduino are Serial Rx & Tx pin Respectively

void setup() {
  //Begin serial communication with Arduino and Arduino IDE (Serial Monitor)
  Serial.begin(9600);

  Serial.println("Initializing Bluetooth Module in Data Mode\n");

  // Begin the soft Serial port and set the data rate for the SoftwareSerial port at 9600 to communicate with
  Bluetooth Module in Data Mode
  BTSerial.begin(9600);

  Serial.print("Bluetooth Module is Successfully Initialized in Data Mode\n");
}

void loop() {
  // Get Data From Serial Terminal and Push to HC-05
  if (Serial.available()) {
    BTSerial.write(Serial.read());
  }

  // Get Data from HC-05 and push to Serial Monitor
  if (BTSerial.available()) {
    Serial.write(BTSerial.read());
  }
}

```

Figura 3: Programa de referencia para la comunicación Bluetooth. [Captura de Pantalla]. s.f. de Play with Circuit (<https://playwithcircuit.com/hc05-bluetooth-module-arduino-tutorial/>)

Se modificó el código para que este:

1. Usara los pines 10 y 11 como los pines RX y TX respectivamente.
2. Pudiera encender y apagar el LED incluido en el Arduino (Pin 13).

A continuación, en la **Figura 4**, se muestra el código modificado.

```

1 #include <SoftwareSerial.h>
2
3 //Se configuran pines RX y TX.
4 SoftwareSerial BTSerial(10, 11);
5
6 //Esta variable guarda el comando enviado
7 char receivedChar;
8
9 void setup() {
10
11   //Configuración de pin 13 (controla el LED integrado en el Arduino)
12   pinMode(13, OUTPUT);
13
14   //Inicialización de comunicación Serial y Bluetooth.
15   Serial.begin(9600);
16   BTSerial.begin(9600);
17 }
18
19 void loop() {
20   // Mientras que el módulo reciba información nueva, el código revisará
21   // si se ha mandado un 0, un 1 o un comando incorrecto y actuará acordemente.
22   while (BTSerial.available() > 0) {
23     receivedChar = BTSerial.read();
24     Serial.print("Mensaje recibido: ");
25     Serial.println(receivedChar);
26
27     if(receivedChar == '1') {
28       Serial.println("Se recibió comando de encendido.");
29       digitalWrite(13, HIGH);
30       BTSerial.println("Se ha encendido el LED.");
31     }
32     else if (receivedChar == '0') {
33       Serial.println("Se recibió comando de apagado.");
34       digitalWrite(13, LOW);
35       BTSerial.println("Se ha apagado el LED.");
36     }
37     else {
38       Serial.println("Se recibió comando inválido.");
39       BTSerial.println("No hubo acción, comando inválido.");
40     }
41   }
42 }
43

```

Figura 4: Código utilizado para la tarea (Autoría Propia).

Se conectó el módulo a otra computadora, la cual contaba con el programa *CoolTerm*, el cual permite comunicarse con el módulo. Si se envía un 1 a través de la terminal de este programa, el LED del Arduino se enciende. Si se envía un 0, el LED se apaga. Cualquier otro comando resultará en un mensaje de error.

Prueba y Error

En el primer intento que se realizó, el módulo logró conectarse a la computadora para mandar y recibir información. Sin embargo, se notó que todo comando enviado (incluso si era 0 o 1) resultaba en un mensaje de error. Al volver a intentar la conexión, se identificó un problema en el módulo dado a que este no podía mantener una conexión con la computadora. Tras varios intentos e investigaciones, se concluyó que el módulo había sido dañado durante su operación. Esto es respaldado por el hecho de que ya no es posible siquiera mandar comandos AT al módulo. Por ende, no se pudo obtener ningún resultado.

Resultados

Debido a las fallas del módulo, no fue posible obtener resultados.

Conclusión

El proyecto representó una oportunidad para aprender más sobre comunicación inalámbrica y lo simple que es conectar un módulo bluetooth a la computadora. Sin embargo, también se identificó la sensibilidad de los módulos y la necesidad de ser precavido para evitar posibles daños. A futuro, se podría mejorar el sistema cambiando el encendido y apagado de un LED por el control de motores o el envío inalámbrico de datos de uno o varios sensores conectados al Arduino.

Progreso de Proyecto Final

Como parte del proyecto final se busca controlar un rover mediante comunicación inalámbrica, por lo cual el siguiente código ha sido diseñado con el objetivo de controlar dos motores DC con base en la señal de Bluetooth recibida.

```
#include <SoftwareSerial.h>

// Bluetooth HC-05
SoftwareSerial Bluetooth(10, 11);
// Pin 10 = RX
// Pin 11 = TX

// Motor izquierdo
int IN1 = 2;
int IN2 = 3;
int ENA = 5;
```

```

// Motor derecho
int IN3 = 4;
int IN4 = 7;
int ENB = 6;

char comando;

void setup() {

    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(ENA, OUTPUT);

    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);
    pinMode(ENB, OUTPUT);

    Bluetooth.begin(9600);
    Serial.begin(9600);

    detener();
}

void loop() {

    if (Bluetooth.available()) {

        comando = Bluetooth.read();

        switch (comando) {

            case 'F':
                adelante();
                break;

            case 'B':
                atras();
                break;

            case 'L':
                izquierda();
                break;

            case 'R':
                derecha();
                break;

            case 'G':

```

```

        adelantelzquierda();
        break;

    case 'I':
        adelanteDerecha();
        break;

    case 'H':
        atraslzquierda();
        break;

    case 'J':
        atrasDerecha();
        break;

    case 'S':
        detener();
        break;
    }
}
}

// -----
// MOVIMIENTOS
// -----

void adelante() {

    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);

    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);

    analogWrite(ENA, 200);
    analogWrite(ENB, 200);
}

void atras() {

    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);

    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);

    analogWrite(ENA, 200);
    analogWrite(ENB, 200);
}

```

```
}
```

```
void izquierda() {
```

```
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, HIGH);
```

```
    digitalWrite(IN3, HIGH);  
    digitalWrite(IN4, LOW);
```

```
    analogWrite(ENA, 200);  
    analogWrite(ENB, 200);
```

```
}
```

```
void derecha() {
```

```
    digitalWrite(IN1, HIGH);  
    digitalWrite(IN2, LOW);
```

```
    digitalWrite(IN3, LOW);  
    digitalWrite(IN4, HIGH);
```

```
    analogWrite(ENA, 200);  
    analogWrite(ENB, 200);
```

```
}
```

```
void adelantelzquierda() {
```

```
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, LOW);
```

```
    digitalWrite(IN3, HIGH);  
    digitalWrite(IN4, LOW);
```

```
    analogWrite(ENA, 0);  
    analogWrite(ENB, 200);
```

```
}
```

```
void adelanteDerecha() {
```

```
    digitalWrite(IN1, HIGH);  
    digitalWrite(IN2, LOW);
```

```
    digitalWrite(IN3, LOW);  
    digitalWrite(IN4, LOW);
```

```
    analogWrite(ENA, 200);  
    analogWrite(ENB, 0);
```

```
}
```

```
void atrasIzquierda() {
```

```
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, LOW);
```

```
    digitalWrite(IN3, LOW);  
    digitalWrite(IN4, HIGH);
```

```
    analogWrite(ENA, 0);  
    analogWrite(ENB, 200);
```

```
}
```

```
void atrasDerecha() {
```

```
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, HIGH);
```

```
    digitalWrite(IN3, LOW);  
    digitalWrite(IN4, LOW);
```

```
    analogWrite(ENA, 0);  
    analogWrite(ENB, 200);
```

```
}
```

```
void detener() {
```

```
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, LOW);
```

```
    digitalWrite(IN3, LOW);  
    digitalWrite(IN4, LOW);
```

```
    analogWrite(ENA, 0);  
    analogWrite(ENB, 0);
```

```
}
```

Referencias

[Circuito de referencia utilizado]. (s.f.). [Play with Circuit].

(<https://playwithcircuit.com/hc05-bluetooth-module-arduino-tutorial/>)

[Código para configurar el módulo]. (s.f.). [Naylamp Mechatronics].

(https://naylampmechatronics.com/blog/24_configuracion-del-modulo-bluetooth-hc-05-usando-comandos-at.html)

¿Cómo funciona la tecnología Bluetooth®?. (s.f.). Intel.

<https://www.intel.la/content/www/xl/es/products/docs/wireless/how-does-bluetooth-work.html>

Configuración del módulo bluetooth HC-05 usando comandos AT. (s.f.). Naylamp Mechatronics.

https://naylampmechatronics.com/blog/24_configuracion-del-modulo-bluetooth-hc-05-usando-comandos-at.html

HC-05 bluetooth module interfacing with arduino. (s.f.). Play with Circuit.

<https://playwithcircuit.com/hc05-bluetooth-module-arduino-tutorial/>

[Programa de referencia para la comunicación Bluetooth]. (s.f.). [Play with Circuit].

(<https://playwithcircuit.com/hc05-bluetooth-module-arduino-tutorial/>)

¿Qué es Bluetooth®?. (s.f.). Lenovo.

<https://www.lenovo.com/mx/es/glosario/bluetooth/?orgRef=https%253A%252F%252Fwww.google.com%252F&srsId=AU7gw4X563xiasWdbRZERlwGwfkhIvaMKGRGp8bMVJT-FzocKzbOfuHg>

Series circuits and the application of Ohm's law. (s.f.). All About Circuits.

<https://www.allaboutcircuits.com/textbook/direct-current/chpt-5/simple-series-circuits/>